

HawkVision Homes — Enterprise Platform Delivery Roadmap

Complete vision, dual-agent engineering model, and production acceptance plan

Scope: Public website, leasing, residents, payments, maintenance, communications, accounting visibility, approvals, compliance, automation, migration, and operations

Current scale: Approximately 100 tenants, designed for portfolio growth

Standard: Production-grade implementation with auditable acceptance—not a prototype or collection of screens

1. Executive intent

HawkVision Homes will receive a unified operating platform spanning the rental lifecycle: listing, inquiry, showing, application, screening, approval, lease, move-in, resident service, payment, maintenance, renewal, move-out, financial oversight, and owner decisions.

This directly addresses the present fragmentation across Innago, Zillow Rental, Facebook, WhatsApp, calls, texts, Cash App exceptions, and Short Landlord. Useful providers may remain during transition, but HawkVision's database becomes the authoritative operational record. Every material action has an owner, state, history, next step, permission boundary, and measurable result.

The outcome is not “another dashboard.” It is a system that routes work, enforces controls, presents exceptions, and helps Farouk focus on the one to three decisions that require owner judgment each day.

2. Business outcomes

- One searchable record for properties, units, prospects, applicants, residents, leases, charges, payments, work orders, vendors, budgets, documents, communications, approvals, and tasks.
- A measurable leasing funnel from lead source to signed lease, including vacancy and conversion reporting.
- One resident experience for balances, payments, receipts, documents, maintenance, messages, and requests.

- Controlled maintenance intake from portal, phone, SMS, and approved legacy channels with assignment, priority, service level, evidence, cost, and completion history.
- A HawkVision-owned resident subledger even when Stripe or another provider moves money.
- Explicit approval for budgets, legal matters, new leases, final tenant selection, and material exceptions until Farouk deliberately delegates authority.
- Consolidation of avoidable subscriptions and manual work without falsely promising that specialized payment, screening, e-signature, accounting, or listing services can all be eliminated.

3. Production reference architecture

- **Experience:** Next.js, TypeScript, responsive web/PWA; public, applicant, resident, staff, vendor, accounting, and owner workspaces.
- **Data and identity:** Supabase PostgreSQL, Auth, Storage, selective Realtime, versioned SQL migrations, and strict row-level security.
- **Application:** Server-only domain services, typed API contracts, background workers, queues, and scheduled jobs.
- **Financial:** Stripe for eligible processing and tokenized methods; HawkVision owns charges, allocations, balances, receipts, adjustments, reversals, and reconciliation.
- **Communications:** Twilio voice/SMS/IVR, approved transactional email, consent and delivery history.
- **Scheduling/documents:** Google or Microsoft calendar, approved screening and e-signature providers, secure object storage, expiring links.
- **Operations:** GitHub, CI/CD, isolated environments, previews, structured logs, error monitoring, analytics, uptime checks, verified backups, and incident runbooks.

Transaction and integration pattern

1. A domain transaction writes the business record and an outbox event together.
2. A worker performs the external action and records each attempt.
3. A signed webhook enters a durable inbox and is deduplicated before processing.
4. Idempotency keys prevent duplicate payments, messages, appointments, and orders.
5. Immutable audit events record actor, action, target, time, source, and material before/after state.
6. Failed work enters a visible retry or exception queue.

This pattern is mandatory for payments, screening, leases, approvals, communications, calendar sync, and work-order routing.

4. Dual-agent delivery model

Emergent: full-stack acceleration lane

Emergent will not be limited to visual scaffolding. It will be used for bounded, end-to-end vertical slices:

- Generate responsive interfaces, database-connected forms, CRUD, dashboards, portals, server APIs, and working workflows.
- Scaffold Supabase tables, Auth, Storage, Realtime behavior, and SQL for engineering review.
- Accelerate Stripe, Twilio, email, calendar, OpenAI, and other supported integrations in sandbox environments.
- Connect approved proprietary or legacy APIs through MCP or custom tools where appropriate.
- Fork bounded features and experiments; create rollback checkpoints before risky changes.
- Use frontend/backend testing subagents, live preview, and pre-deployment health checks.
- Pull and push GitHub branches so every generated change is inspectable and recoverable.

Cursor: engineering control lane

Cursor remains the production engineering control plane:

- Own repository architecture, domain boundaries, shared types, schema migrations, contracts, and configuration.
- Review every Emergent diff; normalize patterns, remove duplication, control dependencies, and reject unsafe output.
- Implement or harden RLS, authorization, financial math, webhook signatures, idempotency, audit, concurrency, queues, rate limits, and recovery.
- Build unit, contract, integration, E2E, RLS, replay, accessibility, load, migration, and restore tests.
- Own observability, CI/CD, environment promotion, release notes, rollback, and production incident readiness.

Git and handoff protocol

- GitHub is canonical. Production is never deployed from an uncommitted agent workspace.
- Branches: `emergent/phase-XX-slice`, `cursor/phase-XX-core`, and `integration/phase-XX`.
- Emergent pulls the approved base, works inside a frozen contract, tests, previews, and pushes a reviewable branch.
- Cursor reviews and hardens the diff, runs the full gates, and merges through the integration branch.

- The tools do not edit the same module concurrently. Schema/API contracts freeze before parallel work.
- Every merge includes requirements, tests, threat/permission review, migration notes, rollback, limitations, and acceptance ownership.

5. Standard phase contract

Every phase uses six checkpoints:

- **A — Functional contract:** Users, states, data, permissions, exceptions, dependencies, and measurable acceptance.
- **B — Emergent build lane:** The complete vertical slice Emergent generates, connects, previews, and tests.
- **C — Cursor engineering lane:** Architecture, invariants, review, hardening, and production implementation.
- **D — Integration, data, and security:** Providers, coexistence, privacy, authorization, audit, and failure recovery.
- **E — Verification evidence:** Retained automated and human proof.
- **F — Acceptance and handoff:** Objective exit gate, owner signoff, documentation, and operating transfer.

No phase is complete because a screen exists or a happy-path demonstration succeeds.

Wave I — Program control and trusted foundation

Phase 1 — Scope, governance, and acceptance authority

1A. Approve the capability register, process inventory, source-of-truth map, exclusions, RACI, change control, risk register, dependencies, and Farouk's reserved decisions.

1B. Emergent creates a navigable cross-role workflow prototype with representative data to validate terminology and operational sequence.

1C. Cursor establishes the monorepo, standards, ADRs, ownership rules, PR controls, backlog linkage, and requirements-to-test traceability.

1D. Inventory provider contracts, API eligibility, credentials, rate limits, exports, webhooks, legal dependencies, and vendor owners; keep unsupported assumptions off the critical path.

1E. Retain signed scope, risk/dependency registers, prototype findings, account-readiness report, and traceability coverage.

1F. Sponsor approves scope, authority, gates, owners, and explicit assumptions; every critical unknown has an owner and resolution date.

Phase 2 — Domain architecture and technical contracts

2A. Define bounded domains, terminology, state machines, data ownership, privacy classes, retention, service boundaries, and nonfunctional requirements.

2B. Emergent builds a property → unit → inquiry → activity-history vertical slice through Supabase and a live preview.

2C. Cursor defines typed domain/API/event/error/authorization contracts and refactors the proof slice into the reference architecture.

2D. Place Stripe, Twilio, calendar, screening, e-signature, email, listings, and accounting behind replaceable adapters.

2E. Produce ADRs, threat model, schema/API contract tests, boundary tests, diagrams, and proof-slice performance results.

2F. Technical lead approves the reference pattern; subsequent phases reuse its transaction, audit, error, event, and authorization controls.

Phase 3 — Environments, CI/CD, secrets, and recovery

3A. Define local, preview, development, staging, and production; release authority; data rules; backup frequency; RPO/RTO; and incident severity.

3B. Emergent configures GitHub-connected previews, health checks, testing subagents, safe checkpoints, and pre-deployment health checks.

3C. Cursor implements lint/type/unit/contract/migration/RLS/E2E gates, protected branches, secret and dependency scanning, SBOMs, and controlled promotion.

3D. Isolate credentials, keep service keys server-only, verify backups and rollback, and document deployment-replacement/data-transfer risk.

3E. Retain clean-install, repeatable migration, failed-release rollback, backup restore, secret scan, and controlled-alert evidence.

3F. A new environment is reproducible from source; staging is isolated; another engineer can execute release and recovery runbooks.

Phase 4 — Design system and multi-role shell

4A. Approve brand, navigation, role information architecture, breakpoints, accessibility, and standard loading/empty/error/permission states.

4B. Emergent builds public, applicant, resident, staff, vendor, accounting, and owner shells plus reusable tables, forms, filters, timelines, dialogs, and mobile navigation.

4C. Cursor consolidates output into a governed component system, typed forms, route guards, server/client boundaries, visual regression, and performance budgets.

4D. Add CSP, secure headers, bot defense, privacy-aware analytics, and assurance that

protected data never enters unauthorized payloads.

4E. Retain keyboard, screen-reader, WCAG 2.2 AA, responsive, browser, Lighthouse, and bundle reports.

4F. All workspaces are consistent and accessible; no module introduces an independent styling, form, or authorization architecture.

Phase 5 — Core data, audit, events, and documents

5A. Define canonical entities, identifiers, relationships, status transitions, provenance, retention, deletion, document classes, and audit scope.

5B. Emergent scaffolds connected administration, secure uploads, search, filters, relationships, activity timelines, and seeded scenarios.

5C. Cursor implements normalized schema, constraints, indexes, optimistic concurrency, immutable audit, outbox/inbox, file metadata, migrations, and factories.

5D. Apply organization/property scope, PII classification, expiring downloads, file validation, encryption where needed, and export/deletion controls.

5E. Retain constraint, concurrency, audit, event replay, retention, query-plan, and representative scenario tests.

5F. Workflows have one truth source; invalid relationships fail; sensitive actions and documents are attributable and recoverable.

Phase 6 — Identity, RBAC, RLS, and delegation

6A. Define public, prospect, applicant, resident, vendor, technician, leasing, manager, accounting, approver, admin, and owner permissions by property and action.

6B. Emergent builds Supabase Auth registration/invitation, verification, reset, profile, session, staff administration, and delegation experiences.

6C. Cursor implements server authorization, PostgreSQL RLS, test harnesses, session controls, privileged MFA readiness, expiring delegation, and separation of duties.

6D. Block cross-resident/property/vendor access, audit permission changes and sensitive reads, and prohibit a shared master code.

6E. Retain deny-by-default matrix, direct API tests, expired session/invitation tests, delegation expiry, and escalation review.

6F. Every role completes authorized work and cannot retrieve or mutate unrelated records through UI, API, or guessed identifiers.

Wave II — Public experience and leasing

Phase 7 — Public website and managed content

- 7A.** Approve sitemap, copy, conversion paths, tenant support routes, legal pages, SEO, content ownership, and publishing workflow.
- 7B.** Emergent builds Home, About, Rentals, Details, Apply, Resources, Maintenance, Contact, FAQ, Privacy, Terms, and staff content management.
- 7C.** Cursor hardens content models, metadata, caching, images, structured data, server forms, analytics events, rate limits, and performance budgets.
- 7D.** Route inquiries into CRM with consent/source data; protect forms from abuse and drafts/internal information from exposure.
- 7E.** Retain crawl, mobile/browser, form delivery, SEO, accessibility, spam, and performance reports.
- 7F.** Staff safely update content; visitors reach a property or correct support route; every conversion is measurable.

Phase 8 — Property, unit, media, and availability

- 8A.** Define fields, pricing, fees, deposits, amenities, utilities, pet/accessibility rules, media, turnover dependencies, availability, and publication rules.
- 8B.** Emergent builds full-stack property/unit/media administration, status flows, preview, bulk edit, duplicate, publish, archive, and responsive record views.
- 8C.** Cursor enforces state transitions, validation, media processing, search indexes, concurrency, publication snapshots, and cache invalidation.
- 8D.** Preserve Zillow/Facebook coexistence and mapping; prevent internal notes, resident data, costs, and unpublished units from leaking.
- 8E.** Retain state, stale-availability, simultaneous-edit, media, public snapshot, and administrative UAT evidence.
- 8F.** Staff update a unit once and approved public surfaces show authorized current information with provenance.

Phase 9 — Listings, search, and channel distribution

- 9A.** Define search, listing readiness, fair-housing content controls, publication cadence, syndication eligibility, reconciliation, and fallback.
- 9B.** Emergent builds rental discovery, filters, detail views, saved searches if justified, publication queue, channel status, and exception screens.
- 9C.** Cursor implements optimized queries, canonical listing payloads, adapters, idempotent publication jobs, drift detection, retry, and rate-limit handling.
- 9D.** Verify actual Zillow, Facebook, and future feed/API rights; never automate unsupported account behavior or promise unavailable APIs.
- 9E.** Retain relevance, payload contract, duplicate publication, outage, accessibility, and reconciliation evidence.
- 9F.** HawkVision owns listing truth; supported channels visibly reconcile; unsupported channels have an explicit staff procedure.

Phase 10 — Inquiry capture and leasing CRM

- 10A.** Define lead sources, consent, deduplication, stages, assignments, SLAs, next action, archive reasons, and KPI definitions.
- 10B.** Emergent builds forms, lead profiles, pipeline, assignments, notes, tags, tasks, timeline, search, saved views, and mobile workflows.
- 10C.** Cursor implements identity resolution, duplicate suggestions, assignment rules, SLA timers, activity events, bulk safeguards, and governed reporting.
- 10D.** Ingest supported third-party leads, retain source/consent, enforce opt-out, and provide spam and delivery-failure queues.
- 10E.** Retain duplicate, incomplete-lead, concurrency, attribution, notification contract, and leasing-team UAT evidence.
- 10F.** Every valid inquiry has a source, owner, stage, service clock, next action, and complete history.

Phase 11 — Showing scheduler and calendar synchronization

- 11A.** Define availability, buffers, capacity, property/access rules, qualification, reschedule, cancel, no-show, reminder, and override behavior.
- 11B.** Emergent builds property booking, confirmation, reminders, changes, directions, feedback, staff calendar, and mobile day view.
- 11C.** Cursor implements slot locking, time-zone correctness, calendar abstraction, idempotent sync, webhook handling, drift repair, and privacy-safe event content.
- 11D.** Use least-privilege calendar OAuth; handle token expiry, external deletion, provider outages, and replayed callbacks.
- 11E.** Retain double-book race, DST/time-zone, webhook replay, outage, reminder, and prospect/staff UAT evidence.
- 11F.** A prospect can book, change, or cancel while internal and external calendars remain synchronized without duplicates.

Phase 12 — Application and document collection

- 12A.** With qualified review, define lawful fields, household data, income/history, references, disclosures, consent, documents, retention, and completeness.
- 12B.** Emergent builds save/resume, co-applicant invitations, uploads, progress, validation, submission, withdrawal, staff review, and missing-item requests.
- 12C.** Cursor implements immutable submission snapshots, versioned disclosures, secure upload pipeline, household linking, concurrency, and completeness rules.
- 12D.** Connect leads, units, showings, consent, screening, and communications; restrict sensitive data and define purge/export procedures.
- 12E.** Retain household, interrupted-session, malicious-file, direct-access, consent-version, and

applicant/staff UAT tests.

12F. A household submits a traceable application; corrections supplement rather than overwrite the original submission.

Phase 13 — Screening, decision, and adverse action

13A. Select providers and have qualified counsel approve permissible purpose, consent, objective criteria, review, dispute, retention, decisions, and adverse-action procedures.

13B. Emergent builds consent, order/status, normalized result, exception, recommendation, approval, notice, and dispute-support workflows.

13C. Cursor implements the adapter, minimal-result storage, criteria versions, review controls, approval lock, retry safety, and prohibited-field safeguards.

13D. Verify provider callbacks; isolate reports; prohibit autonomous AI housing decisions; preserve notices and Farouk's final approval.

13E. Retain sandbox, replay, provider-outage, criteria-version, permission, legal checklist, and decision-audit evidence.

13F. No final decision or adverse action occurs without authorized human approval, documented basis, required notice, and audit history.

Phase 14 — Lease, e-signature, and move-in

14A. Define approved templates, clauses, signer order, fees/deposits, addenda, approvals, expiry, countersignature, storage, and move-in prerequisites.

14B. Emergent builds lease review/generation, approval queue, e-sign status, reminders, executed access, move-in checklist, keys/utilities, and condition intake.

14C. Cursor implements versioned templates, document hashes, provider adapter, webhook state machine, duplicate protection, activation transaction, and snapshots.

14D. Use expiring links, restrict executed leases, and require Farouk's new-lease approval until expressly delegated.

14E. Retain signer-order, decline/expiry, hash, replay, failed-activation recovery, and end-to-end leasing evidence.

14F. An applicant becomes a resident only after every approval, signature, payment, document, and required task is satisfied.

Wave III — Resident and financial operations

Phase 15 — Resident portal and household hub

- 15A.** Define household access to balance, payments, receipts, documents, maintenance, messages, profile requests, events, and support.
- 15B.** Emergent builds the mobile-first portal, household invitations, activity, documents, payment entry, maintenance, messaging, preferences, and help.
- 15C.** Cursor hardens membership, loaders, cache boundaries, accessible statements, profile approvals, preferences, and low-bandwidth performance.
- 15D.** Enforce lease scope, protect financial/document payloads, audit sensitive access, and prevent cache leakage between accounts.
- 15E.** Retain cross-household denial, mobile/accessibility, statement, session revocation, load, and resident-pilot evidence.
- 15F.** Residents complete routine work without staff and cannot access another household's records.

Phase 16 — Billing rules and resident subledger

- 16A.** Define rent and other charges, proration, concessions, deposits, late fees, grace, partials, allocation, credits, reversals, write-offs, returns, and statements with legal/accounting review.
- 16B.** Emergent builds rule administration, ledger and statement views, adjustment requests, exception queues, and explainable balances.
- 16C.** Cursor implements an append-only double-entry-style journal, deterministic allocation, effective-dated rules, closed periods, reversible corrections, decimals, and balance invariants.
- 16D.** Separate processor events from accounting facts; require privileged approval and reason codes for adjustments; make events immutable and exportable.
- 16E.** Retain golden ledger, property-based balance, proration, partial, return, reversal, concurrency, close, and accountant reconciliation tests.
- 16F.** Every balance reconstructs from journal entries; corrections preserve history; accounting approves representative statements.

Phase 17 — Stripe payments, autopay, receipts, and reconciliation

- 17A.** Define methods, fees, autopay consent/schedule, partials, failures, refunds, disputes, settlement, receipts, and ownership.
- 17B.** Emergent implements Stripe test-mode tokenized payment/setup flows, autopay UX, receipts, failure recovery, and staff status views.
- 17C.** Cursor hardens secrets, PaymentIntents, signatures, idempotency, event order, duplicate delivery, ledger posting, refunds, disputes, returns, and reconciliation.
- 17D.** Avoid raw card handling; map provider objects to domain records; keep processor and ledger states distinct; create daily exceptions.
- 17E.** Retain success, decline, timeout, retry, replay, delayed event, refund, dispute, return, security, and ledger tie-out evidence.

17F. Money movement is never inferred from the browser; signed provider events reconcile to immutable postings and visible exceptions.

Phase 18 — Cash exceptions and Innago coexistence

18A. Define Cash App, check, cash, Innago, bank transfer, correction, unidentified deposit, and historical balance evidence, approval, and reconciliation.

18B. Emergent builds manual intake, evidence upload, unidentified-payment queue, resident match, transfer/team-note replacement, approval, and dashboards.

18C. Cursor implements controlled posting, deduplication, imports, matching, four-eyes approval where needed, staging, lineage, and balancing reports.

18D. Support phased Innago coexistence; validate exports/APIs; never overwrite HawkVision silently; use the documented \$1,975 Cash App case in UAT.

18E. Retain import dry run, duplicate, out-of-balance, bypass, lineage, exception-aging, and accountant-signoff evidence.

18F. Every channel reaches the ledger through a controlled path; unidentified or unreconciled funds remain visible until resolved.

Wave IV — Maintenance, communications, and controls

Phase 19 — Omnichannel maintenance intake and triage

19A. Define emergencies, safety scripts, categories, severity, response targets, troubleshooting, media, entry permission, routing, and after-hours escalation.

19B. Emergent builds portal/staff intake, mobile media, guided triage, duplicate suggestions, emergency messaging, resident status, assignment, and queue.

19C. Cursor implements priority rules, SLA timers, durable notifications, escalation jobs, rule versions, duplicate linking, safety controls, and event history.

19D. Convert phone/SMS and feasible approved WhatsApp events into tickets; prohibit unsafe AI diagnosis; scope location/resident data to assignments.

19E. Retain emergency/non-emergency, after-hours drill, duplicate, upload-security, notification-failure, and technician/resident UAT evidence.

19F. Every request is owned, prioritized, timed, and auditable; emergencies use an approved human escalation path.

Phase 20 — Work orders, technicians, and vendors

- 20A.** Define assignment, scheduling, access, estimates, limits, parts/labor, resident coordination, status, completion evidence, rework, invoice, warranty, and performance.
- 20B.** Emergent builds mobile technician queue, vendor portal, schedule, notes, before/after media, time/materials, estimates, updates, completion, and invoice upload.
- 20C.** Cursor implements state machine, assignment conflicts, draft recovery, signed uploads, cost rollups, approval triggers, invoice match, warranty links, and metrics.
- 20D.** Limit vendors to assigned work, redact unnecessary resident data, log access, and connect approved costs without conferring payment authority.
- 20E.** Retain cross-vendor denial, reassignment, completion-proof, over-limit estimate, invoice-match, and field UAT with both maintenance workers.
- 20F.** Work is traceable from intake through verified completion, resident communications, cost, approval, invoice, warranty, and rework.

Phase 21 — Communications and Twilio phone tree

- 21A.** Define channels, consent, opt-out, quiet hours, templates, recording/transcription, retention, escalation, and IVR choices: emergency maintenance, standard maintenance, payments, leasing, urgent owner.
- 21B.** Emergent builds Twilio IVR/SMS, shared inbox, timelines, templates, attachments, callbacks, routing, internal notes, and administration.
- 21C.** Cursor implements contact normalization, thread correlation, signatures, idempotency, consent, delivery states, retries/dead letters, redaction, and audit.
- 21D.** Link conversations only to authorized records; validate WhatsApp eligibility; protect payment and maintenance-sensitive information.
- 21E.** Retain IVR, opt-out, quiet-hours, replay, failed-delivery, privacy, emergency-handoff, and staff UAT evidence.
- 21F.** Calls/messages reach an accountable queue and correct history without exposing unrelated resident information.

Phase 22 — Approval, delegation, and exception engine

- 22A.** Define approval classes, thresholds, evidence, separation, expiry, escalation, delegation, emergency override, and reserved owner decisions.
- 22B.** Emergent builds approval inbox, mobile decision cards, evidence, comments, change requests, delegation, escalation, and history.
- 22C.** Cursor implements versioned policy-as-data, thresholds, quorum, atomic locks, signed decisions, conflicts, expiry, and tamper-evident audit.
- 22D.** Trigger controls from leasing, maintenance, budgets, adjustments, legal/compliance, and procurement; never enforce approval only in UI.
- 22E.** Retain boundary, self-approval, expired delegation, concurrent decision, policy version, bypass, override, and export tests.
- 22F.** No reserved or over-threshold action executes without a valid, current, attributable approval.

Phase 23 — Budgets, turns, projects, and procurement

23A. Define property/unit/project budgets, categories, commitments, estimates, change orders, actuals, variance, turnover scope, limits, and accounting mappings.

23B. Emergent builds budget/project workspaces, estimate comparison, commitments, variance, receipts/invoices, approvals, and owner mobile decisions.

23C. Cursor implements effective-dated budgets, aggregation, commitment-versus-actual logic, threshold events, change history, concurrency, and exports.

23D. Link work orders, vendors, invoices, documents, and approvals; preserve accounting classification authority and protect bank/cost details.

23E. Retain the documented shower/turn scenario, over-budget, change-order, concurrency, cost tie-out, export, and owner/accounting UAT.

23F. Farouk sees scope, budget, commitments, actuals, variance, required decision, and evidence before authorization.

Phase 24 — Accounting visibility and Short Landlord

24A. Define chart/property mappings, bank/card feeds, classification, reconciliation, close, P&L, cash, exports, and accountant procedure.

24B. Emergent builds accounting overview, unmatched queue, property P&L, cash/receivable snapshots, reconciliation, exports, and drill-down.

24C. Cursor implements provider/import adapter, mapping rules, source snapshots, matching suggestions, duplicate controls, close locks, lineage, and variance detection.

24D. Validate Short Landlord's real API/export options; govern file exchange if necessary; never label operational estimates as finalized books.

24E. Retain bank/card import, duplicate file, reconciliation, closed period, P&L comparison, lineage, and accountant evidence.

24F. Management receives timely traceable visibility while the accountant and accounting system retain defined authority.

Phase 25 — Documents, insurance, renewals, and compliance

25A. Define document classes, owners, confidentiality, expiry/renewal, retention, legal hold, evidence, reminders, and escalation.

25B. Emergent builds document vault, metadata, relationships, versions, expiry dashboard, renewal workflow, acknowledgments, calendar, and missing-item queue.

25C. Cursor implements secure storage, checksums, versioning, expiring links, retention jobs, reminders, escalations, access logs, and recovery.

25D. Connect policies, properties, vendors, leases, staff, and tasks; isolate legal/identity files and durably track delivery.

25E. Retain expiry simulation, access denial, version integrity, reminder retry, retention/legal

hold, access audit, and restore tests.

25F. Every required document/deadline has an owner, state, evidence, reminder, and escalation before lapse.

Wave V — Command center, intelligence, and launch

Phase 26 — Daily command center, tasks, and reporting

26A. Define daily decisions, KPI formulas, service levels, drill-down, task generation, exception aging, ownership, and freshness.

26B. Emergent builds role home screens, Farouk's one-to-three decision queue, portfolio exceptions, leasing/vacancy, collections, maintenance, approvals, tasks, and summaries.

26C. Cursor implements governed metrics, read models, freshness, lineage, task deduplication, priority scoring, optimized queries, and correct exports.

26D. Use canonical records, distinguish estimated/final financial data, restrict portfolio/PII views, and alert on stale pipelines.

26E. Retain KPI catalog, source tie-outs, stale-data simulation, role tests, load/performance, and owner/staff UAT.

26F. Each number is defined and drillable; every decision card shows context, amount/risk, action, deadline, and owner.

Phase 27 — Rules automation, AI, MCP, and controlled agents

27A. Rank automations by value/risk; define allowed actions, confidence, review, data boundaries, prompt/model versions, cost, fallback, and prohibitions.

27B. Emergent uses custom agents, MCP, integrations, and subagents for message drafts, inquiry triage, maintenance categorization, extraction, task suggestions, summaries, and exception preparation.

27C. Cursor implements allowlisted tools, structured outputs, retrieval boundaries, injection defenses, redaction, evaluations, fallback, cost telemetry, approval checkpoints, and audit.

27D. Agents use authorized domain APIs; they cannot finalize tenant decisions, legal matters, leases, budgets, payments, ledger adjustments, or emergency diagnosis.

27E. Retain curated evaluations, hallucination/unsafe-action, prompt-injection, permission, cost/latency, override, and quality-review evidence.

27F. Every automation has a business owner, measurable benefit, monitored error rate, kill switch, fallback, and human accountability.

Phase 28 — Migration, security, launch, adoption, and stabilization

28A. Define source inventories, cleansing, mappings, waves, dual-entry/freeze, communications, training, support, go/no-go, hypercare, and retirement criteria.

28B. Emergent builds migration correction/review, onboarding, guided tours, help, import status, release communications, support intake, and training environment.

28C. Cursor implements repeatable importers, dry runs, checksums, reconciliation, rollback, load tests, monitoring, alerts, runbooks, flags, and controlled promotion.

28D. Complete tenant isolation, OWASP review, secret/dependency scans, restore, incident tabletop, production credentials, processing register, and legacy access reduction.

28E. Retain entity/dollar reconciliation, regression, payment replay, RLS, accessibility, load/soak, vulnerability, restore, UAT, training, and launch evidence.

28F. Go-live requires zero critical defects, reconciled control totals, proven rollback, live monitoring, staffed support, trained users, and named stabilization ownership.

6. Cross-program production gates

Domain	Required evidence
Functional correctness	Requirements trace to tests; all critical journeys pass end-to-end in staging
Authorization	Positive/negative RBAC and RLS suites; direct API isolation tests
Financial integrity	Reconstructable ledger, deterministic allocation, idempotent posting, reconciliation, accountant signoff
Integration resilience	Signed webhooks, inbox dedupe, idempotency, retry/dead-letter queues, outage exercises
Security	Threat model, secret/dependency scans, secure headers, rate limits, least privilege, vulnerability review
Privacy	PII inventory, retention, access logs, consent/opt-out, export/deletion procedure
Accessibility	WCAG 2.2 AA review for critical journeys; keyboard, screen-reader, contrast, and error validation
Performance	Journey budgets, representative load tests, slow-query review, monitored production percentiles

Reliability	Agreed availability, health checks, alert ownership, tested restore, documented RPO/RTO
Operations	Runbooks, dashboards, incident roles, escalation contacts, rollback, release/change history
Adoption	Staff/resident pilots, training, help, support, adoption metrics, phased retirement criteria

Initial targets, to be confirmed after vendor/hosting selection: 99.9% monthly availability for the core portal excluding announced maintenance; p95 server response under 500 ms for ordinary authenticated reads at normal load; public Core Web Vitals in the “good” range at p75; RPO of 24 hours or better and RTO of four hours or better for core operations; immediate alerting for payment and authentication failures.

7. Required evidence package per phase

1. Approved functional contract and acceptance cases.
2. Updated schema, API, event, state, and permission contracts.
3. Emergent branch, implementation summary, preview, and testing-subagent results.
4. Cursor review identifying accepted, rewritten, or rejected generated code and why.
5. Migration, configuration, secrets, deployment, and rollback notes.
6. Automated report plus retained evidence for critical journeys.
7. Threat/permission review proportional to risk.
8. Known limitations, runbook changes, owner, and support impact.
9. Product, technical, and business acceptance signoff.

8. Seven-day sprint positioning

The 28 phases define the complete production vision. They are not a claim that a secure, integrated, migrated enterprise platform can responsibly be proven complete in seven calendar days by one person. A seven-day intensive sprint can produce a compelling working vertical slice, production architecture, validated provider proofs, priority workflows, test scaffolding, and a credible implementation baseline. Production acceptance still depends on provider access, legal/accounting decisions, data quality, security evidence, user testing, reconciliation, and stabilization.

The fastest responsible approach is to use Emergent aggressively for bounded full-stack slices, integrations, previews, and tests while Cursor continuously protects contracts, migrations, security, financial integrity, and repository quality. Demonstrate public listing → inquiry → showing → application, plus resident payment and maintenance experiences early; keep financial, legal, identity, screening, and cutover gates evidence-driven.

9. Definition of complete

HawkVision is complete only when approved users execute the agreed production workflows; integrations fail safely; financial and sensitive actions are reconstructable; permissions withstand direct access attempts; migration reconciles; monitoring and recovery are proven; staff and residents are trained; operating runbooks are transferred; and every gate has retained evidence and an accountable signatory.

That is the standard appropriate to a six-figure implementation: not the number of generated screens, but the reliability, control, adoption, traceability, and business outcomes of the operating system as a whole.